Module 3 Structured Note: Counting Vowels, Consonants, Words, and Spaces

1. What this module is about

Module 3 teaches how to scan a C-style string and count different things inside it.

You will learn how to count:

1. Vowels
2. Consonants
3. Words
4. Spaces and word boundaries
5. Words correctly even when there are extra spaces

The main skill is:

```
Start from index 0.
Check one character at a time.
Update counters when needed.
Stop when you reach '\0'.
```

This is the same basic pattern from Module 2, but now we are counting characters instead of changing them.

---

2. Required idea from previous modules

Before Module 3, remember this:

A C-style string is a character array ending with the null terminator `\0`.

Example:

```
char A[] = "How are you";
```

Memory view:

```
Index:  0 1 2 3 4 5 6 7 8 9 10 11
Value:  H o w   a r e   y o u  \0
```

The visible text is:

```
How are you
```

The `\0` is not part of the sentence. It only tells C where the string ends.

So most string loops look like this:

```c
for (i = 0; A[i] != '\0'; i++) {
    // process A[i]
}
```

This means:

```
Start at the first character.
Keep going while the current character is not \0.
Stop when the string ends.
```

Part A: Counting vowels and consonants

3. What are vowels?

The vowels are:

```
a e i o u
A E I O U
```

We check both lowercase and uppercase because C treats `'a'` and `'A'` as different characters.

Example:

```c
if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] == 'u' ||
    A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] == 'U') {
```

```
    vcount++;
}
```

Meaning:

```
If the current character is a vowel, increase the vowel count.
```

---

4. What are consonants?

A consonant is an alphabet letter that is not a vowel.

Examples of consonants:

```
b c d f g h j k l m n p q r s t v w x y z
B C D F G H J K L M N P Q R S T V W X Y Z
```

But these are not consonants:

```
space
1
2
3
!
@
#
```

This is important because a consonant is not simply “anything that is not a vowel.”

Wrong idea:

```
If it is not a vowel, it must be a consonant.
```

Correct idea:

```
If it is a vowel, count it as a vowel.
Else if it is an alphabet letter, count it as a consonant.
Else ignore it.
```

---

5. Why we need an alphabet check

Suppose the character is `'!'`.

It is not a vowel, but it is also not a consonant.

So after checking for vowels, we must check whether the character is actually an alphabet letter.

A character is an uppercase letter if it is between:

```
'A' and 'Z'
```

A character is a lowercase letter if it is between:

```
'a' and 'z'
```

So the alphabet check is:

```
(A[i] >= 'A' && A[i] <= 'Z') || (A[i] >= 'a' && A[i] <= 'z')
```

Meaning:

```
The character is either uppercase A-Z or lowercase a-z.
```

---

6. Counter variables

To count vowels and consonants, we use two counters:

```
int vcount = 0;
int ccount = 0;
```

Meaning:

```
vcount stores the number of vowels.
ccount stores the number of consonants.
```

Both start from `0` because before scanning the string, we have not found anything yet.

7. Basic logic for vowel and consonant counting

The logic is:

```
For each character:
    If it is a vowel:
        increase vcount
    Else if it is an alphabet letter:
        increase ccount
    Else:
        ignore it
```

In C-style form:

```
for (i = 0; A[i] != '\0'; i++) {
    if (A[i] is vowel) {
        vcount++;
    }
    else if (A[i] is alphabet letter) {
        ccount++;
    }
}
```

---

8. Code: Count vowels and consonants

```
#include <stdio.h>

int main() {
    char A[] = "How are you! 123";
    int i;
    int vcount = 0;
    int ccount = 0;

    for (i = 0; A[i] != '\0'; i++) {

        if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] ==
'u' ||
            A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] ==
'U') {
            vcount++;
        }
```

```
        else if ((A[i] >= 'A' && A[i] <= 'Z') ||
                 (A[i] >= 'a' && A[i] <= 'z')) {
            ccount++;
        }
    }

    printf("Vowels: %d\n", vcount);
    printf("Consonants: %d\n", ccount);

    return 0;
}
```

Output:

```
Vowels: 5
Consonants: 4
```

---

9. Dry run: `"How are you! 123"`

String:

```
How are you! 123
```

Character-by-character trace:

Character	Type	Action
`H`	consonant	`ccount++`
`o`	vowel	`vcount++`
`w`	consonant	`ccount++`
space	not alphabet	ignore
`a`	vowel	`vcount++`
`r`	consonant	`ccount++`
`e`	vowel	`vcount++`
space	not alphabet	ignore
`y`	consonant	`ccount++`
`o`	vowel	`vcount++`

Character	Type	Action
`u`	vowel	`vcount++`
`!`	symbol	ignore
space	not alphabet	ignore
`1`	digit	ignore
`2`	digit	ignore
`3`	digit	ignore
`\0`	string end	stop

Final count:

```
Vowels = 5
Consonants = 4
```

Vowels found:

```
o, a, e, o, u
```

Consonants found:

```
H, w, r, y
```

Part B: Counting words using spaces

10. What is a word?

For this module, a simple word is a group of characters separated by spaces.

Example:

```
How are you
```

Words:

```
How
are
you
```

So the word count is:

```
3
```

---

11. Simple word counting idea

In a clean sentence, words are separated by one space.

Example:

```
How are you
```

There are two spaces:

```
How are you
   ^   ^
```

There are three words:

```
How | are | you
```

So for a clean sentence:

```
words = spaces + 1
```

If spaces = 2, then:

```
words = 2 + 1 = 3
```

---

12. Simple word count code

```c
#include <stdio.h>

int main() {
    char A[] = "How are you";
    int i;
    int word = 1;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] == ' ') {
            word++;
        }
    }

    printf("Words: %d\n", word);

    return 0;
}
```

Output:

```
Words: 3
```

Why does `word` start from `1`?

Because if a sentence has no spaces but has one word, the word count should still be `1`.

Example:

```
Hello
```

Spaces:

```
0
```

Words:

```
1
```

So the simple method starts with:

```
int word = 1;
```

Then it adds one word for every space.

---

13. Dry run: simple word count for `"How are you"`

String:

```
How are you
```

Initial value:

```
word = 1
```

Trace:

Character	Is it a space?	Action	word
`H`	No	ignore	1
`o`	No	ignore	1
`w`	No	ignore	1
space	Yes	`word++`	2
`a`	No	ignore	2
`r`	No	ignore	2
`e`	No	ignore	2
space	Yes	`word++`	3
`y`	No	ignore	3
`o`	No	ignore	3
`u`	No	ignore	3
`\0`	End	stop	3

Final answer:

```
Words = 3
```

Part C: The problem with extra spaces

14. Why the simple word count can fail

The simple method counts every space as a word separator.

That works here:

```
How are you
```

But it fails here:

```
How  are you
```

There are two spaces between `How` and `are`.

Visual view:

```
How  are you
   ^^   ^
```

The simple method sees three spaces, so it calculates:

```
words = 3 spaces + 1 = 4
```

But the real words are:

```
How
are
you
```

Correct answer:

```
3
```

So we need improved logic.

---

15. Main idea for handling extra spaces

Instead of counting every space, we should count only the spaces that actually separate words.

A space separates words only when:

```
Current character is a space
and
Previous character is not a space
```

In C:

```c
if (A[i] == ' ' && A[i - 1] != ' ') {
    word++;
}
```

This prevents repeated spaces from increasing the word count again and again.

---

16. Improved word count using previous character

```c
#include <stdio.h>

int main() {
    char A[] = "How  are you";
    int i;
    int word = 1;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] == ' ' && A[i - 1] != ' ') {
            word++;
        }
    }

    printf("Words: %d\n", word);
```

```
    return 0;
}
```

Output:

```
Words: 3
```

Important warning:

This version has a problem when the string starts with a space.

Example:

```
char A[] = " How are you";
```

At `i = 0`, checking `A[i - 1]` means checking `A[-1]`, which is invalid.

So this version needs a guard:

```
if (i > 0 && A[i] == ' ' && A[i - 1] != ' ') {
    word++;
}
```

But even with that guard, it may still count badly if the sentence begins or ends with spaces.

For beginners, a cleaner robust method is the `inWord` method.

Part D: Better word counting using `inWord`

17. What does `inWord` mean?

The `inWord` variable tells us whether we are currently inside a word.

```
int inWord = 0;
```

Meaning:

```
inWord = 0 means we are outside a word.
inWord = 1 means we are inside a word.
```

This method does not count spaces.

Instead, it counts when a new word begins.

---

18. Logic of the `inWord` method

For each character:

```
If the current character is not a space and we are outside a word:
    This is the start of a new word.
    Increase word count.
    Set inWord = 1.

Else if the current character is a space:
    We are outside a word.
    Set inWord = 0.
```

In simple words:

```
Count a word only when you enter it.
Do not count every space.
```

---

19. Robust word count code

```c
#include <stdio.h>

int main() {
    char A[] = "  How   are you  ";
    int i;
    int word = 0;
    int inWord = 0;

    for (i = 0; A[i] != '\0'; i++) {

        if (A[i] != ' ' && inWord == 0) {
            word++;
            inWord = 1;
```

```
        }
        else if (A[i] == ' ') {
            inWord = 0;
        }
    }

    printf("Words: %d\n", word);

    return 0;
}
```

Output:

```
Words: 3
```

This method handles:

```
"How are you"
"How  are you"
"  How are you"
"How are you  "
"  How   are   you  "
```

Correctly.

---

20. Dry run: robust word count for `"  How   are you  "`

Initial values:

```
word = 0
inWord = 0
```

String:

```
  How   are you
```

Trace:

Character	Meaning	Action	word	inWord
space	outside word	stay outside	0	0
space	outside word	stay outside	0	0
`H`	start of word	`word++`, enter word	1	1
`o`	inside word	no new word	1	1
`w`	inside word	no new word	1	1
space	end of word	leave word	1	0
space	outside word	stay outside	1	0
space	outside word	stay outside	1	0
`a`	start of word	`word++`, enter word	2	1
`r`	inside word	no new word	2	1
`e`	inside word	no new word	2	1
space	end of word	leave word	2	0
`y`	start of word	`word++`, enter word	3	1
`o`	inside word	no new word	3	1
`u`	inside word	no new word	3	1
space	end of word	leave word	3	0
space	outside word	stay outside	3	0
`\0`	string end	stop	3	0

Final answer:

```
Words = 3
```

Part E: Required dry run from the module: `"w  a"`

21. Previous-character word count dry run

The module uses the example:

```
w  a
```

There are two spaces between `w` and `a`.

The correct word count is:

```
2
```

Words:

```
w
a
```

Using previous-character logic:

```
if (A[i] == ' ' && A[i - 1] != ' ') {
    word++;
}
```

Start:

```
word = 1
```

Trace:

i	A[i]	A[i - 1]	Action	word
0	`w`	none	not a space	1
1	space	`w`	previous is not space, so `word++`	2
2	space	space	previous is also space, ignore	2
3	`a`	space	not a space	2
4	`\0`	`a`	stop	2

Final answer:

```
Words = 2
```

Why this works:

The first space after `w` marks a boundary between words. The second space is just an extra space, so it should not create another word.

Part F: Counting spaces

22. Simple space counting

If you only want to count how many spaces are in a string, check for `' '`.

Code:

```
#include <stdio.h>

int main() {
    char A[] = "How are you";
    int i;
    int spaces = 0;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] == ' ') {
            spaces++;
        }
    }

    printf("Spaces: %d\n", spaces);

    return 0;
}
```

Output:

```
Spaces: 2
```

This counts all spaces, including repeated spaces.

Example:

```
How  are you
```

Spaces:

```
3
```

Words:

```
3
```

So space count and word count are not always directly connected when there are extra spaces.

Part G: Full Module 3 combined program

23. Count vowels, consonants, words, and spaces

```c
#include <stdio.h>

int main() {
    char A[] = "  How   are you! 123  ";
    int i;

    int vcount = 0;
    int ccount = 0;
    int spaces = 0;
    int word = 0;
    int inWord = 0;

    for (i = 0; A[i] != '\0'; i++) {

        // Count vowels
        if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] ==
'u' ||
            A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] ==
'U') {
            vcount++;
        }

        // Count consonants
        else if ((A[i] >= 'A' && A[i] <= 'Z') ||
                 (A[i] >= 'a' && A[i] <= 'z')) {
            ccount++;
        }

        // Count spaces
        if (A[i] == ' ') {
```

```
            spaces++;
        }

        // Count words using inWord method
        if (A[i] != ' ' && inWord == 0) {
            word++;
            inWord = 1;
        }
        else if (A[i] == ' ') {
            inWord = 0;
        }
    }

    printf("Vowels: %d\n", vcount);
    printf("Consonants: %d\n", ccount);
    printf("Spaces: %d\n", spaces);
    printf("Words: %d\n", word);

    return 0;
}
```

Output:

```
Vowels: 5
Consonants: 4
Spaces: 8
Words: 4
```

Why words are `4` here:

```
How
are
you!
123
```

This simple word-counting method treats any non-space group as a word.

So `you!` is one word, and `123` is also counted as one word.

---

24. Important note about word definitions

There are different ways to define a word.

In this module, the simple definition is:

```
A word is any group of non-space characters.
```

So this counts as one word:

```
123
```

And this also counts as one word:

```
you!
```

But if you want a stricter definition, such as:

```
A word must contain alphabet letters.
```

Then the word-count logic must be changed.

For now, Module 3 focuses on space-based word counting.

Part H: Safer version for alphabet-only words

25. Optional improvement: Count only alphabetic words

Sometimes we may not want to count `123` as a word.

For that, we can define a word as a group that starts with an alphabet letter.

Code:

```c
#include <stdio.h>

int main() {
    char A[] = "  How   are you! 123  ";
    int i;
    int word = 0;
    int inWord = 0;

    for (i = 0; A[i] != '\0'; i++) {
```

```
        int isAlphabet = (A[i] >= 'A' && A[i] <= 'Z') ||
                         (A[i] >= 'a' && A[i] <= 'z');

        if (isAlphabet && inWord == 0) {
            word++;
            inWord = 1;
        }
        else if (A[i] == ' ') {
            inWord = 0;
        }
    }

    printf("Alphabetic words: %d\n", word);

    return 0;
}
```

This is an extra improvement. The main module method is still the space-based method.

Part I: Complexity analysis

26. Time complexity

For vowel and consonant counting, we scan the string once.

If the string has `N` characters, the loop visits each character one time.

So time complexity is:

```
O(N)
```

For word counting, we also scan the string once.

So word count time complexity is also:

```
O(N)
```

---

27. Space complexity

The programs use only a few variables:

```
int i;
int vcount;
int ccount;
int word;
int spaces;
int inWord;
```

They do not create a second array.

So extra space complexity is:

```
O(1)
```

This means constant extra space.

The amount of extra memory does not grow when the string becomes longer.

---

28. Complexity table

Operation	Time Complexity	Extra Space
Count vowels	`O(N)`	`O(1)`
Count consonants	`O(N)`	`O(1)`
Count spaces	`O(N)`	`O(1)`
Simple word count	`O(N)`	`O(1)`
Improved word count	`O(N)`	`O(1)`

Part J: Common mistakes

29. Mistake 1: Counting every non-vowel as a consonant

Wrong:

```
if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] == 'u') {
    vcount++;
}
else {
    ccount++;
}
```

Why this is wrong:

The `else` also counts spaces, digits, and symbols as consonants.

Example:

```
How are you! 123
```

This wrong logic would count:

```
space, !, 1, 2, 3
```

as consonants, which is incorrect.

Correct:

```
else if ((A[i] >= 'A' && A[i] <= 'Z') ||
         (A[i] >= 'a' && A[i] <= 'z')) {
    ccount++;
}
```

---

30. Mistake 2: Checking only lowercase vowels

Wrong:

```
if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] == 'u')
```

This misses uppercase vowels:

```
A E I O U
```

Better:

```
if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] == 'u' ||
    A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] == 'U')
```

---

31. Mistake 3: Counting every space as a new word

This works for:

```
How are you
```

But fails for:

```
How   are you
```

Wrong result:

```
5 words
```

Correct result:

```
3 words
```

Use previous-character logic or the `inWord` method to handle extra spaces.

---

32. Mistake 4: Accessing `A[i - 1]` when `i = 0`

This is risky:

```
if (A[i] == ' ' && A[i - 1] != ' ')
```

At `i = 0`, this checks:

```
A[-1]
```

That is invalid.

Safer:

```
if (i > 0 && A[i] == ' ' && A[i - 1] != ' ')
```

Even better for beginners:

```
Use the inWord method.
```

---

33. Mistake 5: Forgetting to stop at `\0`

Wrong:

```
for (i = 0; i < 100; i++)
```

Better:

```
for (i = 0; A[i] != '\0'; i++)
```

Why?

Because the string may end before the full array capacity.

The null terminator tells us the real end of the string.

Part K: Main code patterns to remember

34. Pattern 1: Scan a string

```
for (i = 0; A[i] != '\0'; i++) {
    // process A[i]
}
```

---

35. Pattern 2: Check vowel

```
if (A[i] == 'a' || A[i] == 'e' || A[i] == 'i' || A[i] == 'o' || A[i] == 'u' ||
    A[i] == 'A' || A[i] == 'E' || A[i] == 'I' || A[i] == 'O' || A[i] == 'U') {
    vcount++;
}
```

---

36. Pattern 3: Check alphabet letter

```
if ((A[i] >= 'A' && A[i] <= 'Z') ||
    (A[i] >= 'a' && A[i] <= 'z')) {
    // alphabet letter
}
```

---

37. Pattern 4: Count consonants

```
else if ((A[i] >= 'A' && A[i] <= 'Z') ||
         (A[i] >= 'a' && A[i] <= 'z')) {
    ccount++;
}
```

This should usually come after the vowel check.

---

38. Pattern 5: Count spaces

```
if (A[i] == ' ') {
    spaces++;
}
```

---

39. Pattern 6: Simple word count

```
int word = 1;

for (i = 0; A[i] != '\0'; i++) {
    if (A[i] == ' ') {
```

```
        word++;
    }
}
```

Use this only when the sentence has exactly one space between words and no leading or trailing spaces.

---

40. Pattern 7: Improved word count with previous character

```
int word = 1;

for (i = 0; A[i] != '\0'; i++) {
    if (i > 0 && A[i] == ' ' && A[i - 1] != ' ') {
        word++;
    }
}
```

This handles repeated spaces better, but still needs care with leading and trailing spaces.

---

41. Pattern 8: Robust word count with `inWord`

```
int word = 0;
int inWord = 0;

for (i = 0; A[i] != '\0'; i++) {
    if (A[i] != ' ' && inWord == 0) {
        word++;
        inWord = 1;
    }
    else if (A[i] == ' ') {
        inWord = 0;
    }
}
```

This is the safest beginner-friendly method in this module.

Part L: Practice questions

42. Practice 1: Count vowels and consonants

Input:

```
How are you! 123
```

Expected answer:

```
Vowels = 5
Consonants = 4
```

Ignored characters:

```
spaces, !, 1, 2, 3
```

---

43. Practice 2: Simple word count

Input:

```
How are you
```

Question:

How many words?

Answer:

```
3
```

Reason:

There are two spaces, so:

```
words = 2 + 1 = 3
```

44. Practice 3: Extra spaces

Input:

```
How   are you
```

Question:

Why does the simple word-count method fail?

Answer:

Because it counts every space as a new word boundary, including repeated spaces.

Correct answer:

```
3 words
```

---

45. Practice 4: Dry run `"w  a"`

Input:

```
w  a
```

Expected word count:

```
2
```

Reason:

The first space separates `w` and `a`. The second space is extra and should be ignored.

---

46. Practice 5: Fix the consonant-counting mistake

Wrong code:

```
if (A[i] is vowel) {
    vcount++;
}
else {
    ccount++;
}
```

Question:

Why is this wrong?

Answer:

Because the `else` counts digits, spaces, and symbols as consonants.

Correct idea:

```
Only alphabet letters that are not vowels should be counted as consonants.
```

Part M: Final summary

47. Key points to remember

1. Module 3 uses the same string traversal pattern from Module 2.
2. Start from index `0` and stop at `\0`.
3. Vowels are `a, e, i, o, u` and `A, E, I, O, U`.
4. Consonants are alphabet letters that are not vowels.
5. Spaces, digits, and symbols are not consonants.
6. Use alphabet range checks before counting consonants.
7. The simple word-count rule is `words = spaces + 1`.
8. The simple word-count rule fails when there are extra spaces.
9. Previous-character logic avoids counting repeated spaces.
10. The `inWord` method is a safer way to count words.
11. Counting vowels, consonants, spaces, and words all take `O(N)` time.
12. These algorithms use `O(1)` extra space.

---

48. Final mental model

For vowel and consonant counting:

```
Look at one character.
If it is a vowel, increase vowel count.
Else if it is an alphabet letter, increase consonant count.
Else ignore it.
Move to the next character.
Stop at \0.
```

For word counting:

```
Do not blindly count every space.
Count when a real new word begins.
Use inWord to know whether you are already inside a word.
```

Final takeaway:

```
Module 3 is about classification and counting.
You scan the string once, decide what each character is, and update the correct
counter.
```